

Tutorial 3

SC1006, CZ1106

qq. 3.1 and 3.2

qq. 3.3 and 3.4 – self study based on the provided suggested solution

3.1 Stack Manipulation Operations

These questions review the ARM instructions and addressing modes that can be used to implement various stack manipulation operations. Student should have a good understanding of the various stack operations.

Fig. 3.1 shows the contents on the system stack and the current position of the stack pointer (**SP**) of an ARM processor. With reference to Fig. 3.1, answer the following questions.

(1) What is the current content in the register **SP**?

Ans: **SP = 0xFEC.**

(2) Give instruction to push the content in **R1** onto the stack.

Ans: **Either STR R1, [SP, #-4]!**
Or STMFD SP!, {R1}

STMFD SP!, {R6-R9}

(3) Give a single instruction to pop off four word-sized items from the stack to registers R6-R9.

Ans: **LDMFD SP!, {R6-R9}.**

<https://azeria-labs.com/functions-and-the-stack-part-7/>

(4) Retrieve 3rd word-sized data on the stack into **R0**.

Ans: **LDR R0, [SP, #8].**

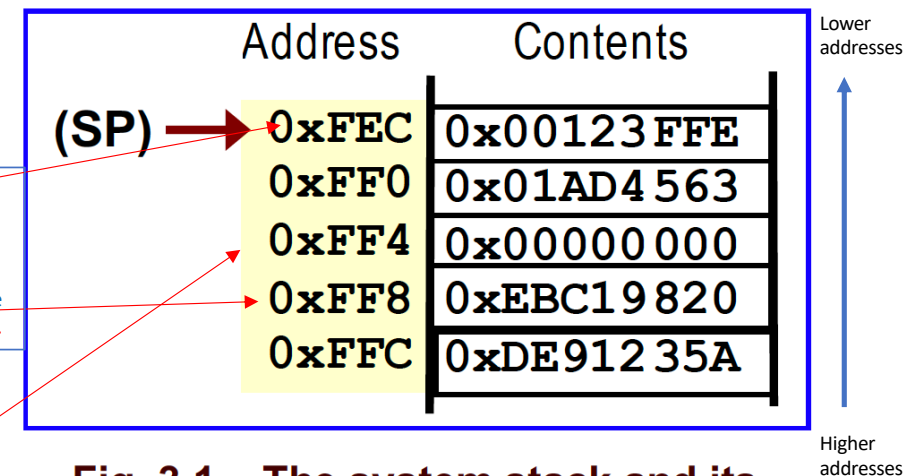


Fig. 3.1 – The system stack and its contents

Stack Type	Store	Load
Full descending	STMFD (STMDB, Decrement Before)	LDMFD (LDM, Increment after)
Full ascending	STMFA (STMIB, Increment Before)	LDMFA (LDMDB, Decrement After)
Empty descending	STMED (STMDA, Decrement After)	LDMED (LDMIB, Increment Before)
Empty ascending	STMEA (STM, Increment after)	LDMEA (LDMDB, Decrement Before)

3.2 Modular Programming – Subroutine Call and Parameter Passing

Fig. 3.2 shows a code segment for subroutine **MySub** and the calling program that makes use of it. With reference to Fig. 3.2, answer the following questions:

- (1) How is each of the parameters **NumX**, **NumY** and **Ans** passed into the **MySub** subroutine? Is it by value or by reference? Give reasons for each of your answers.
- (2) With the aid of the code address shown in Fig. 3.2, give the hexadecimal content of the **PC** and **SP** immediately after the execution of each of the instructions at (b1), (c1) and (s6). Assume execution begins at instruction (a1).
- (3) What is the hexadecimal content in **R0** after instruction (d1) is executed?
- (4) Give a single ARM instruction at (e1) that would remove all the parameters pushed to the stack?
- (5) Replace the **BL MySub** instruction with a branch and any other supporting instruction that supports branching to subroutine.
- (6) With the help of the comments given, complete the subroutine **MySub**. This subroutine implements the function **Ans = NumX * Num Y**. The unsigned word-sized values of **NumX** and **NumY** are multiplied and the resulting word-sized multiplication is stored directly to memory variable **Ans**. For example, given the values of 0x004 and 0x003 for **NumX** and **NumY** respectively, the subroutine **MySub** should compute the result 0x00C and place this result into the memory variable **Ans** as shown in Fig. 3.2.

Code Address				Address	Contents
0x000	Start	MOV	SP, #0xFFFFF0FC ; (a1) Initialize stack pointer		
0x004		MOV	R0, #0x100	NumX	0x100 0x004
0x008		LDR	R1, [R0]	NumY	0x104 0x003
0x00C		STR	R1, [SP, #-4]! ; (b1) NumX to stack	Ans	0x108 0x00C
0x010		ADD	R0, R0, #4		
0x014		LDR	R1, [R0]		
0x018		STR	R1, [SP, #-4]! ; (b2) NumY to stack		
0x01C		ADD	R0, R0, #4		
0x020		STR	R0, [SP, #-4]! ; (b3) Ans to stack		
0x024		BL	MySub ; (c1)		
		LDR	R0, [SP, #-4] ; (d1)		
		?	?		
			;		
			;		
0x040	MySub	?	?		
		?	?		
		?	?		
		:			
		:			
		?	?		
		?	?		
		?	?		
		MOV	PC, LR		

Fig. 3.2 – A partially completed ARM assembly language program

Code Address					Address	Contents
0x000	Start	MOV	SP, #0xFFFFFEC	; (a1) Initialize stack pointer		
0x004		MOV	R0, #0x100		NumX	0x100
0x008		LDR	R1, [R0]		NumY	0x104
0x00C		STR	R1, [SP, #-4]!	; (b1) NumX to stack	Ans	0x108
0x010		ADD	R0, R0, #4			0x10C
0x014		LDR	R1, [R0]			0x110
0x018		STR	R1, [SP, #-4]!	; (b2) NumY to stack		
0x01C		ADD	R0, R0, #4			
0x020		STR	R0, [SP, #-4]!	; (b3) Ans to stack		
0x024		BL	MySub	; (c1)		
		LDR	R0, [SP, #-4]	; (d1)		
		?	?	; (e1) Remove stack parameters		
		:				

(1) How is each of the parameters **NumX**, **NumY** and **Ans** passed into the **MySub** subroutine? Is it by value or by reference? Give reasons for each of your answers.

Suggested solutions:

NumX and **NumY** are passed by value since the actual values of these memory variables are pushed to the stack using the absolute addressing mode.

Ans is passed using reference since the address of the memory variable **Ans** is pushed onto the stack using immediate addressing.

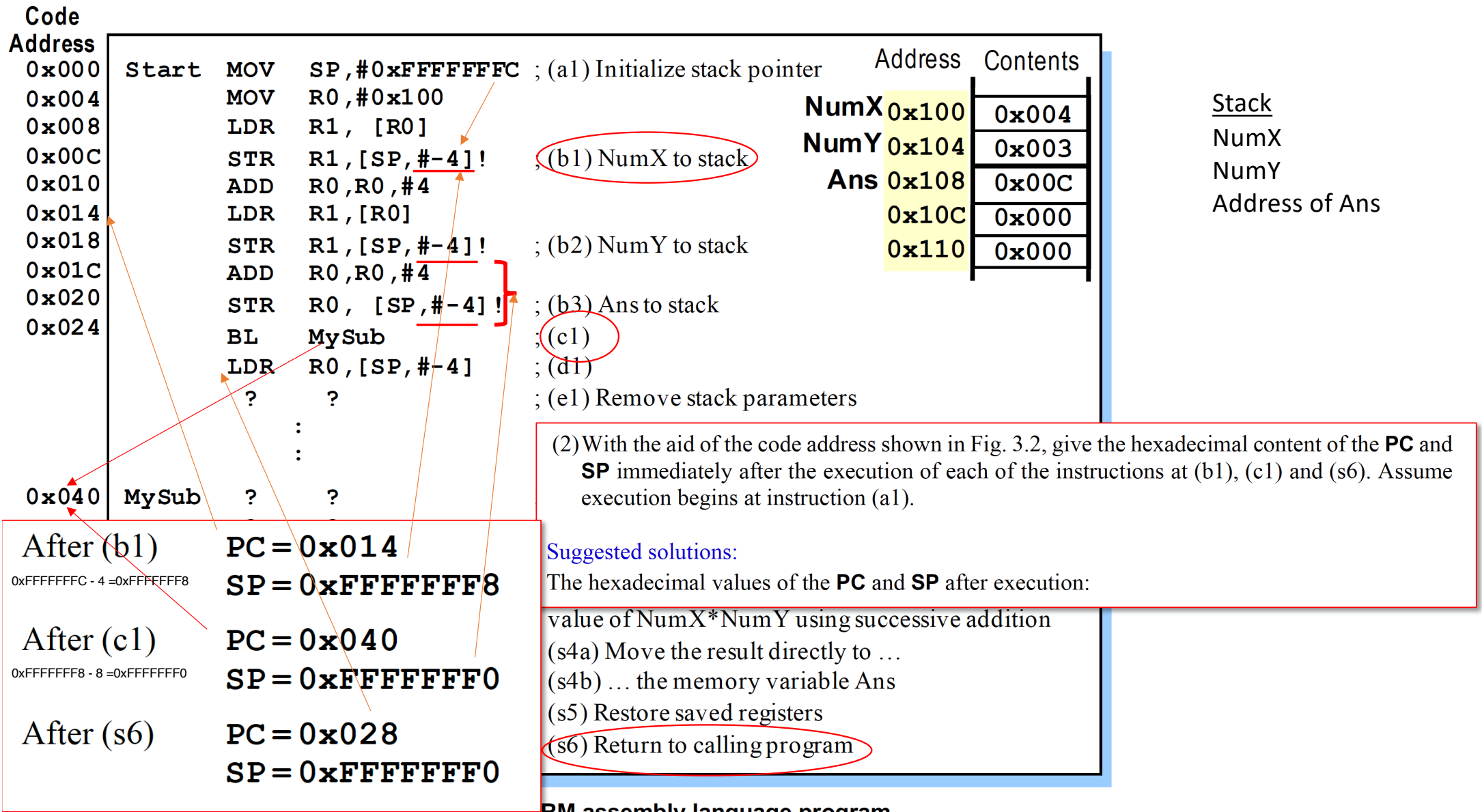
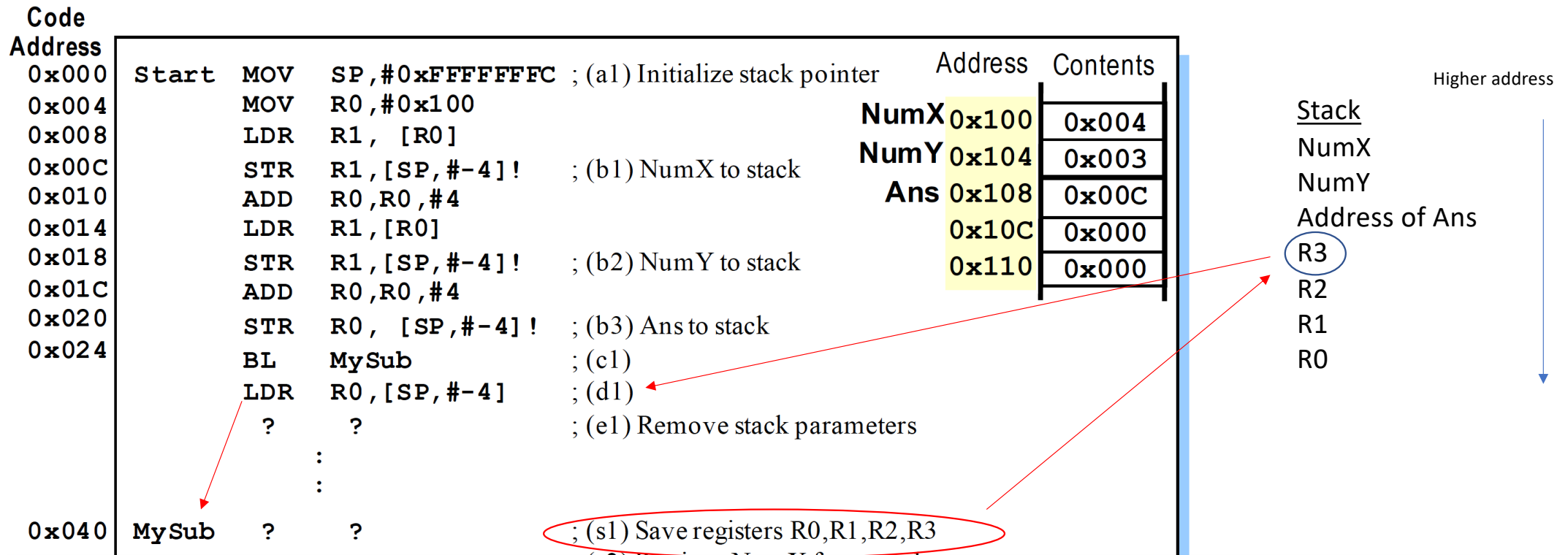


Fig. 3.2 – A partially completed ARM assembly language program



(3) What is the hexadecimal content in **R0** after instruction (d1) is executed?

Suggested solutions:

R0=the value of R3 (can be 0). The subroutine pushed R0,R1,R2,R3 to the stack where R3 is placed right above Ans. IF we assume R3 has 0 then R0 would then have 0.

Fig. 3.2 – A partially completed ARM assembly language program

Code Address					Address	Contents	
0x000	Start	MOV	SP, #0xFFFFFFFF	; (a1) Initialize stack pointer			Stack (SP+12)
0x004		MOV	R0, #0x100		NumX	0x100	0x004
0x008		LDR	R1, [R0]		NumY	0x104	0x003
0x00C		STR	R1, [SP, #-4]!	; (b1) NumX to stack	Ans	0x108	0x00C
0x010		ADD	R0, R0, #4			0x10C	0x000
0x014		LDR	R1, [R0]			0x110	0x000
0x018		STR	R1, [SP, #-4]!	; (b2) NumY to stack			
0x01C		ADD	R0, R0, #4				
0x020		STR	R0, [SP, #-4]!	; (b3) Ans to stack			
0x024		BL	MySub	; (c1)			
		LDR	R0, [SP, #-4]	; (d1)			
		?	?	; (e1) Remove stack parameters			

(4) Give a single ARM instruction at (e1) that would remove all the parameters pushed to the stack?

Suggested solutions:

ADD SP, SP, #12. Three word-size parameters are push to the stack. Moving the stack pointer by adding $3 \times 4 = 12$ to its content can allow us to remove or pop these items.

?	?	; (s5) Restore saved registers
MOV PC, LR		; (s6) Return to calling program

Fig. 3.2 – A partially completed ARM assembly language program

Code Address					Address	Contents	
0x000	Start	MOV	SP, #0xFFFFFFFF	; (a1) Initialize stack pointer			B label causes a branch to label.
0x004		MOV	R0, #0x100		NumX	0x100	0x004
0x008		LDR	R1, [R0]		NumY	0x104	0x003
0x00C		STR	R1, [SP, #-4]!	; (b1) NumX to stack	Ans	0x108	0x00C
0x010		ADD	R0, R0, #4			0x10C	0x000
0x014		LDR	R1, [R0]			0x110	0x000
0x018		STR	R1, [SP, #-4]!	; (b2) NumY to stack			
0x01C		ADD	R0, R0, #4				
0x020		STR	R0, [SP, #-4]!	; (b3) Ans to stack			
0x024		BL	MySub	; (c1)			
		LDR	R0, [SP, #-4]	; (d1)			
		?					
			(5) Replace the BL MySub instruction with a branch and any other supporting instruction that supports branching to subroutine				
			Replacement would be as follows				
0x040	MySub	?					
		?					
		?	MOV LR, PC	LR (or R14) is used as a <i>link register</i> to store the return address when a subroutine call is made			
		?	B MySub				
		?					
		?					
		?					
			MOV PC, LR	; (s6) Return to calling program			

Fig. 3.2 – A partially completed ARM assembly language program

(6) With the help of the comments given, complete the subroutine **MySub**. This subroutine implements the function **Ans = NumX * NumY**. The unsigned word-sized values of **NumX** and **NumY** are multiplied and the resulting word-sized multiplication is stored directly to memory variable **Ans**. For example, given the values of 0x004 and 0x003 for **NumX** and **NumY** respectively, the subroutine **MySub** should compute the result 0x00C and place this result into the memory variable **Ans** as shown in Fig. 3.2.

0x014	LDR	R1, [R0]	
0x018	STR	R1, [SP, #-4]!	; (b2) NumY to stack
0x01C	ADD	R0, R0, #4	
0x020	STR	R0, [SP, #-4]!	; (b3) Ans to stack
0x024	BL	MySub	; (c1)
	LDR	R0, [SP, #-4]	; (d1)
	?	?	; (e1) Remove stack parameters
	:		
	:		
0x040	MySub	?	?
		?	?

Stack	
NumX (SP+24)	
NumY (SP+20)	
Address of Ans (SP+16)	
R3 (SP+12)	
R2 (SP+8)	
R1 (SP+4)	
R0 (SP)	

partially completed ARM assembly language program

0x10C	4 Start	MOV	SP, #0xFFFFFCC
0x110	5	MOV	R0, #0x100
	6	LDR	R1, [R0]
	7	STR	R1, [SP, #-4]!
	8	ADD	R0, R0, #4
	9	LDR	R1, [R0]
	10	STR	R1, [SP, #-4]!
	11	ADD	R0, R0, #4
	12	STR	R0, [SP, #-4]!
	13	BL	MySub
	14	LDR	R0, [SP, #-4]
	15	ADD	SP, SP, #12
	16	end	
	17 MySub	STMFD	SP!, {R0-R3}
	18	LDR	R0, [SP, #24]
	19	LDR	R1, [SP, #20]
	20	MOV	R2, #0
	21 Loop	ADD	R2, R2, R1
	22	SUBS	R0, R0, #1
	23	BNE	Loop
	24	LDR	R3, [SP, #16]
	25	STR	R2, [R3]
	26	LDMFD	SP!, {R0-R3}
	27	Mov	PC, LR

Fig. A3.2 – Completed (and tested) ARM assembly language program

```

17 MySub    STMFD    SP!, {R0-R3}
18          LDR      R0, [SP, #24]
19          LDR      R1, [SP, #20]
20          MOV      R2, #0
21 Loop     ADD      R2, R2, R1
22          SUBS     R0, R0, #1
23          BNE      Loop
24          LDR      R3, [SP, #16]
25          STR      R2, [R3]
26          LDMFD    SP!, {R0-R3}
27          Mov      PC, LR

```

	Address	Contents
NumX	0x100	0x004
NumY	0x104	0x003
Ans	0x108	0x00C
	0x10C	0x000
	0x110	0x000

$R2 = \text{NumY} + \text{NumY} + \dots + \text{NumY} \quad \{\text{NumX times}\}$

Points to note:

1. The appropriate offsets from SP required to access the various parameters are shown in the stack memory map in Fig. A3.2. An offset of +24 is needed to get to NumX, +20 for NumY and +16 for the address of Ans on the stack.
2. This subroutine can be written with fewer registers that is shown. If register R1 is re-used after the successive addition as the pointer register for memory variable Ans, then register R3 is not necessary.
3. A simple count loop using the initial value of NumY is used to perform successive addition of NumX to a register that started with a value of 0. This essentially implements the multiplication of $\text{NumX} * \text{NumY}$.

3.3 Generating Time Delays using Software

Fig. 3.3 shows a subroutine for generating a short time delay.

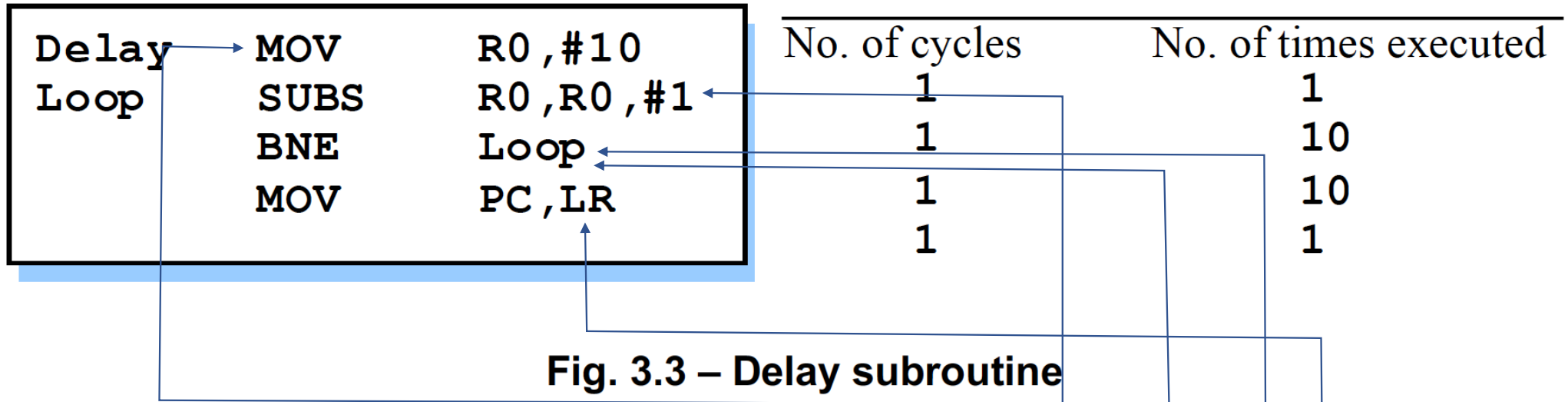
- (1) Calculate the delay produced by the subroutine in terms of number of execution cycles.
- (2) Given that the ARM processor system clock is 10MHz, what is the duration of the time delay produced by the routine? You may assume that each memory access cycle uses one cycle of the system clock.
- (3) How can a time delay of 1ms (millisecond) be achieved?

Delay	MOV	R0 ,#10
Loop	SUBS	R0 ,R0 ,#1
	BNE	Loop
	MOV	PC ,LR

Fig 3.3 – Delay subroutine

This question shows how short and precise time delays can be generated using appropriate loop structures. It should be highlighted that in practical microprocessor systems, it is more likely that hardware timers with appropriate interrupts are used to generate precise time delays as such software methods can be very power hungry and prevents the processor from doing anything else that is useful during the generation of the time delay.

3.3 Generating Time Delays using Software



(1) Calculate the delay produced by the subroutine in terms of number of execution states.

Ans: Tallying up the total number of cycles will give $(1 + 1 \times 10 + 3 \times 9 + 1 + 3) = 42$ cycles.

The BNE consumes 3 cycles when branch is taken, 1 cycle if not taken

(2) Given that the processor clock is 10MHz, what is the duration of the time delay produced by the routine?

Ans: At 10MHz, each cycle is $0.1 \mu\text{s}$, therefore total delay = $42 \times 0.1 = 4.2 \mu\text{s}$
 4.2 microsecond

$1/10,000,000 = 0.1$ microsecond

3.3 Generating Time Delays using Software

Fig. 3.3 shows a subroutine for generating a short time delay.

(3) How can a time delay of 1ms be achieved? 1 ms = 0.001 sec = 1000 microsec

Ans: Delay can be increased by putting a larger value into R0 to increase the number of loops. A delay of 1ms will be generated by $(1 \times 10^{-3} * 10 \times 10^6) = 10000$ clock cycles if the clock is 10 MHz. This value can be obtained by solving the equation: $2 + (4 \times N) = 10000$. Solving for N gives **2499 (0x9C3)**. So we change **MOV R0, #10** to **MOV R0, #0x9C0**, then **ADD R0, R0, #0x03**. (the last statement can be adding 2, as the new instruction consumes one more cycle).

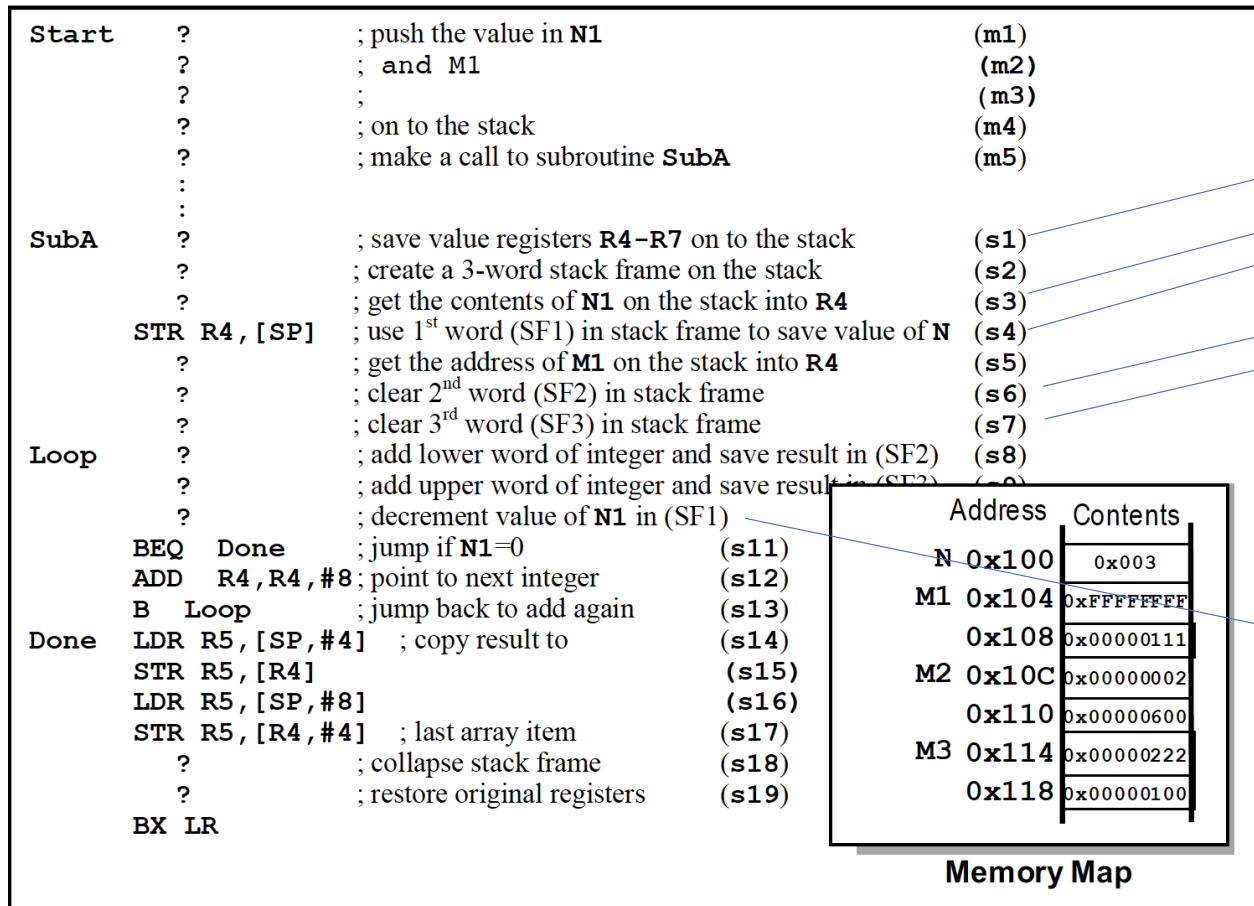
42 cycles = $4 * 10 + 2$ for 10 loops, therefore for N loops it is $4 * N + 2$

$N = (10000 - 2) / 4 = 2499.5 \approx$ approximately 0x9C3

0x9C0 can be obtained as an immediate value, the offset 3 is required then

3.4 Multi-precision Arithmetic

An ARM assembly language program and the contents of several memory variables are given in Fig. 3.4. Each multi-precision integer **M1**, **M2** and **M3** is stored using two words in memory and in Little Endian format. The memory variable **N** stores the number of two-word integers in the array. On completion of the multi-precision addition routine, the last two-word integer in the array is replaced by the total sum of all the two-word integers in the array.



```

1 N1      DCD      0x3
2 M1      DCD      0xFFFFFFFF, 0x111
3 M2      DCD      0x2, 0x600
4 M3      DCD      0x222, 0x100
5
6 START   MOV      SP, #0xFFFFFCC
7         LDR      R0, =N1
8         LDR      R1, [R0]
9         ADD      R0, R0, #4
10        STMFD    SP!, {R1, R0}
11        BL       SubA
12        END
13
14 SubA   STMFD    SP!, {R4-R7}
15        ADD      SP, SP, #-12
16        LDR      R4, [SP, #32]
17        STR      R4, [SP]
18        LDR      R4, [SP, #28]
19        MOV      R5, #0
20        STR      R5, [SP, #4]
21        STR      R5, [SP, #8]
22 Loop   LDR      R5, [SP, #4]
23        LDR      R6, [R4], #4
24        LDR      R7, [R4], #4
25        ADDS     R6, R5, R6
26        LDR      R5, [SP, #8]
27        ADC      R7, R7, R5
28        STR      R6, [SP, #4]
29        STR      R7, [SP, #8]
30        LDR      R7, [SP]
31        SUBS     R7, R7, #1
32        STR      R7, [SP]
33        BNE      Loop
34        LDR      R5, [SP, #4]
35        LDR      R6, [SP, #8]
36        STR      R5, [R4, #-8]
37        STR      R6, [R4, #-4]
38        ADD      SP, SP, #12
39        LDMFD    SP!, {R4-R7}
40        MOV      PC, LR
  
```

ADC:
Add with Carry
adds two
register values
and the Carry
flag value
and writes the
result to the
destination
register.

Fig 3.4 – An incomplete ARM calling program and subroutine

(2) Describe what changes to the program in Fig. 3.4 you would make if the multi-precision integers are stored using the Big Endian format instead.

Ans: Since Big Endian has the lower word stored at the higher address, the higher address must be added first using ADD. This can be done by changing lines 22-29 as follows:

```
LDR R5, [SP, #8]
LDR R6, [R4, #4]
LDR R7, [R4]
ADDS R6, R6, R5
LDR R5, [SP, #4]
ADC R7, R7, R5
STR R6, [SP, #8]
STR R7, [SP, #4]
```

```
1 N1      DCD      0x3
2 M1      DCD      0xFFFFFFFF, 0x111
3 M2      DCD      0x2, 0x600
4 M3      DCD      0x222, 0x100
5
6 START   MOV      SP, #0xFFFFFEC
7         LDR      R0, =N1
8         LDR      R1, [R0]
9         ADD      R0, R0, #4
10        STMFD    SP!, {R1, R0}
11        BL       SubA
12        END
13
14 SubA    STMFD    SP!, {R4-R7}
15        ADD      SP, SP, #-12
16        LDR      R4, [SP, #32]
17        STR      R4, [SP]
18        LDR      R4, [SP, #28]
19        MOV      R5, #0
20        STR      R5, [SP, #4]
21        STR      R5, [SP, #8]
22 Loop   LDR      R5, [SP, #4]
23        LDR      R6, [R4], #4
24        LDR      R7, [R4], #4
25        ADDS     R6, R5, R6
26        LDR      R5, [SP, #8]
27        ADC      R7, R7, R5
28        STR      R6, [SP, #4]
29        STR      R7, [SP, #8]
30        LDR      R7, [SP]
31        SUBS     R7, R7, #1
32        STR      R7, [SP]
33        BNE      Loop
34        LDR      R5, [SP, #4]
35        LDR      R6, [SP, #8]
36        STR      R5, [R4, #-8]
37        STR      R6, [R4, #-4]
38        ADD      SP, SP, #12
39        LDMFD    SP!, {R4-R7}
40        MOV      PC, LR
```


(3) Give the two 12-bit hexadecimal values in memory addresses 0x105 and 0x106 at the end of the execution of the ARM code segment shown in Fig 3.4.

Ans: 0x114=0x223 (lower word) and
0x118=0x812 (upper word)

This is obtained with the following addition:

```

0x111 FFFFFFFF
0x600 0000000002
+0x100 0000000222
-----
0x812 0000000223
-----

```

712 0000000001

	Address	Contents
N	0x100	0x003
M1	0x104	0xFFFFFFFF
	0x108	0x00000111
M2	0x10C	0x00000002
	0x110	0x00000600
M3	0x114	0x00000222
	0x118	0x00000100

```

1 N1      DCD      0x3
2 M1      DCD      0xFFFFFFFF,0x111
3 M2      DCD      0x2,0x600
4 M3      DCD      0x222,0x100
5
6 START   MOV      SP, #0xFFFFF0C
7         LDR      R0, =N1
8         LDR      R1, [R0]
9         ADD      R0, R0, #4
10        STMFD    SP!, {R1, R0}
11        BL       SubA
12        END
13
14 SubA    STMFD    SP!, {R4-R7}
15        ADD      SP, SP, #-12
16        LDR      R4, [SP, #32]
17        STR      R4, [SP]
18        LDR      R4, [SP, #28]
19        MOV      R5, #0
20        STR      R5, [SP, #4]
21        STR      R5, [SP, #8]
22 Loop    LDR      R5, [SP, #4]
23        LDR      R6, [R4], #4
24        LDR      R7, [R4], #4
25        ADDS     R6, R5, R6
26        LDR      R5, [SP, #8]
27        ADC      R7, R7, R5
28        STR      R6, [SP, #4]
29        STR      R7, [SP, #8]
30        LDR      R7, [SP]
31        SUBS     R7, R7, #1
32        STR      R7, [SP]
33        BNE      Loop
34        LDR      R5, [SP, #4]
35        LDR      R6, [SP, #8]
36        STR      R5, [R4, #-8]
37        STR      R6, [R4, #-4]
38        ADD      SP, SP, #12
39        LDMFD    SP!, {R4-R7}
40        MOV      PC, LR

```